NOVA SCHOOL OF SCIENCE & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

MEI - Master in Computer Science and Engineering

Internet Applications Design and Implementation

Week 5 - Foundations of Software Security. Spring Security.

Foundations of Software Security

Main concepts used in software security

- **Principal** (the active subject)
- **Resource** (the passive object)
- **Authentication** - the principal provides proof that they are who they say they are
- **Authorisation** - the principal is trusted to access or perform an operation on a resource
- **Delegation** - can you operate on behalf of other principal?

© Pedro Alves 2026

Principal/Authentication

- An authenticated entity (or group) that uses the system
- The active subject on system operations
- **Authentication**:
  - The certification process of determining the identity of an external entity that uses the system, invokes actions, produces and consumes data.
  - Typically, the identity can be based on one or more of the following:
    - Something the subject knows (e.g., password)
    - Something the subject has (e.g., a smartcard)
    - Something the subject is (e.g. biometric identification)

© Pedro Alves 2026

Resource/Authorization

- **Resources** a principal is allowed to access
- **Operations** a principal is allowed to execute at a given time and system state
- **Authorisation** (to observe or change)
  - read, execute, write, append, delete, change access, change ownership, …
  - access control models
    - Access Control Matrix
    - Access Control Lists
    - Capabilities
    - Role-based Access Control
    - Model-based Access Control

© Pedro Alves 2026

Access Control Lists (ACLs)

- ACLs are lists of permissions (identity, operation) attached to objects stored within the system
- It must explicitly assign individual identities to operations on resources

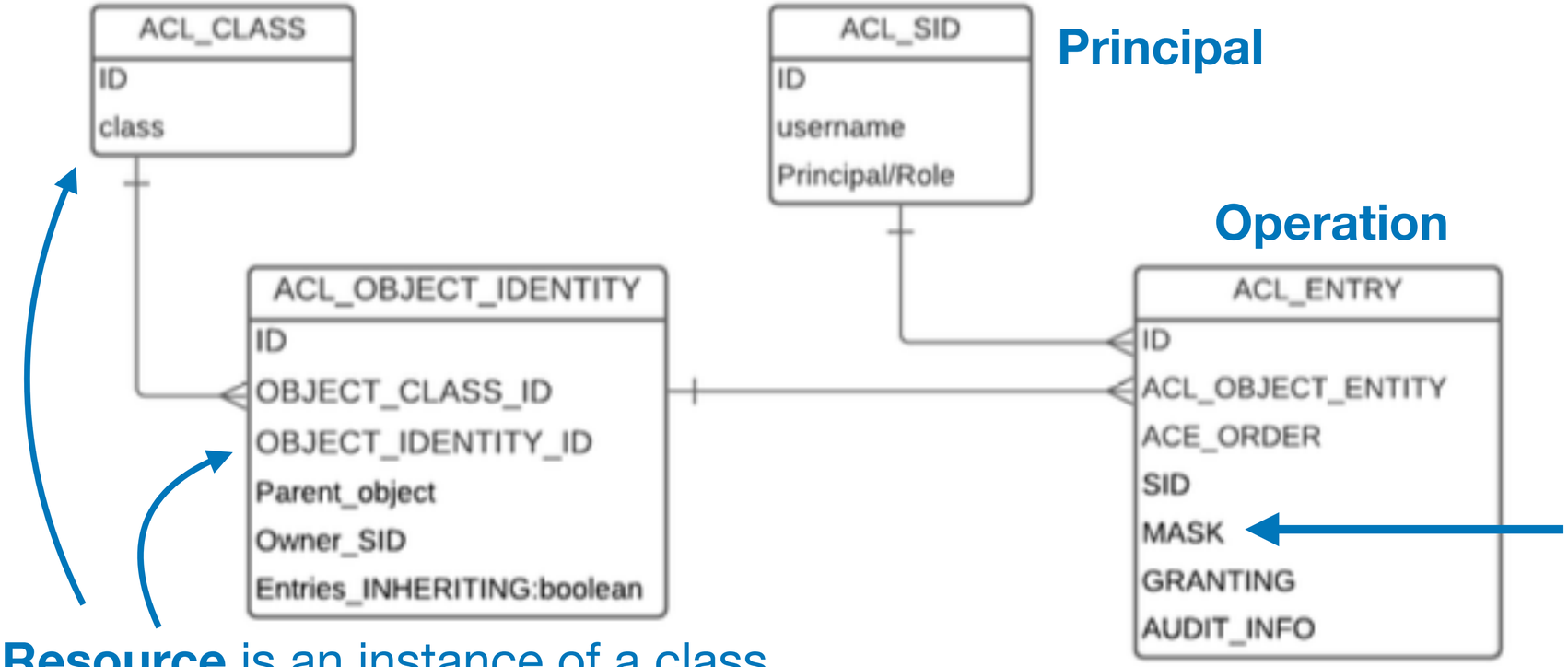
```
(user3,CREATE,obj1)
(user1,READ,obj1)
(user2,READ,obj1)
(user2,WRITE,obj1)
```

© Pedro Alves 2026

Authorization

Spring Security ACL

Spring Security ACL implements ACL on top of explicitly managed database tables



Permission	mask value	bits
READ	1	00000001
WRITE	2	00000010
CREATE	4	00000100
DELETE	8	00001000
ADMINISTRATION	16	00010000

Resource is an instance of a class

© Pedro Alves 2026

Authorization

Spring Security ACL

Example: BlogPost

1. Create ACL for each instance of BlogPost

```
private fun grantDefaultPermissions(postId: Long) {
    val oi = ObjectIdentityImpl(BlogPost::class.java, postId)
    val acl = aclService.createAcl(oi)
    acl.insertAce(0, BasePermission.READ,           PrincipalSid("user1"),   true)
    acl.insertAce(1, BasePermission.WRITE,          PrincipalSid("user2"), true)
    acl.insertAce(2, BasePermission.DELETE,         PrincipalSid("user3"),  true)
    acl.insertAce(3, BasePermission.ADMINISTRATION, PrincipalSid("user3"),  true)
    aclService.updateAcl(acl)
}
```

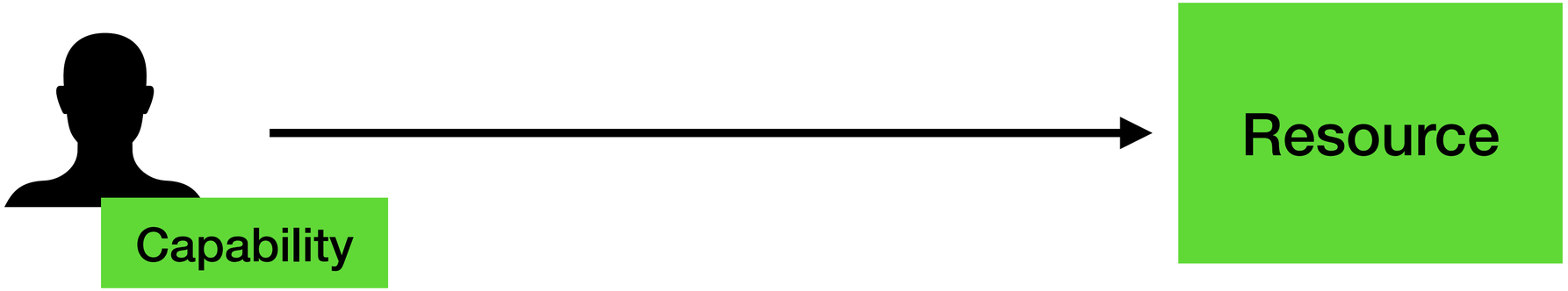
2. Check permission through @PreAuthorize annotation

```
@PreAuthorize("hasPermission(#id, 'pt.unl.fct.iadi.springacl.BlogPost', 'write')")
fun update(id: Long, title: String, content: String): BlogPost {
    val post = blogPostRepository.findById(id).orElseThrow()
    return blogPostRepository.save(post.copy(title = title, content = content))
}
```

© Pedro Alves 2026

Capability-based access control

- A capability refers to an object and a set of rights
  - Users that own it can perform the operation described in the capability
  - Example: a pair (“/etc/pass”, O_RDWR) gives readwrite access
- Decentralised method <-> scalable approach to access control



© Pedro Alves 2026

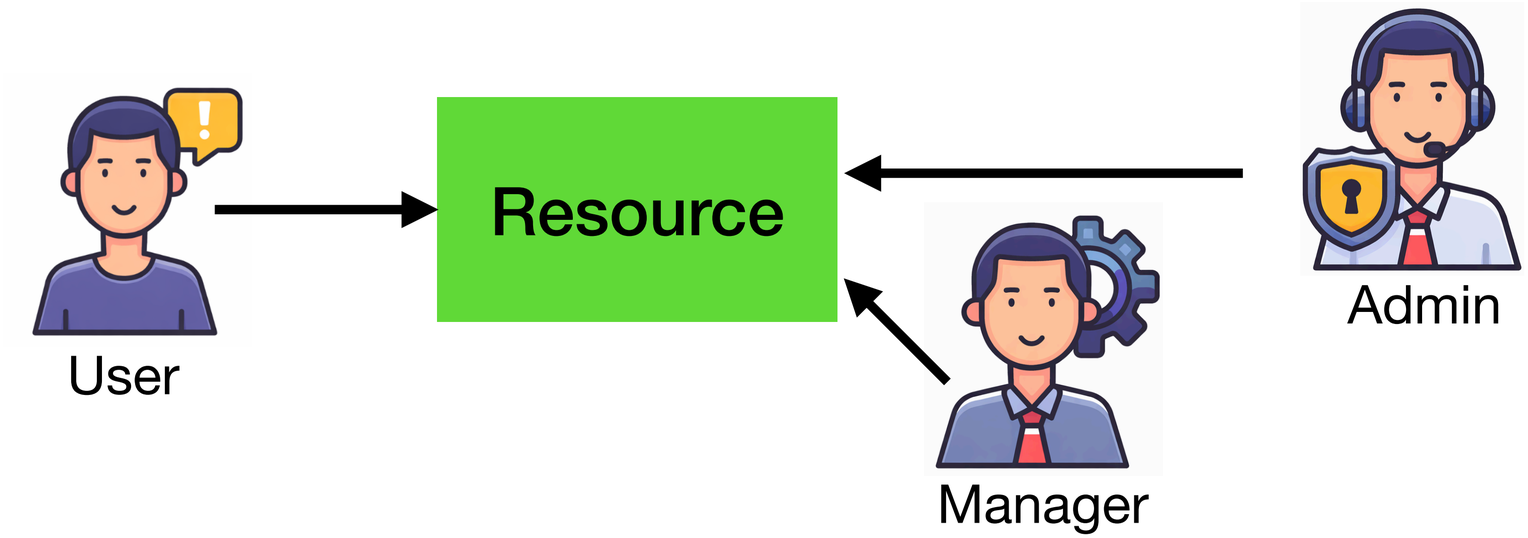
Capability-based access control

- A capability is transmitted through unforgeable (and ephemeral) tokens
- Capability-based systems are inherently decentralized, as authorization is not centrally enforced by a global authority (e.g., ACLs or role-based systems).
- Instead, possession of a capability is itself sufficient authority, eliminating the need to consult a central access control service.
- Capabilities can be delegated directly between principals, allowing authority to flow dynamically through the system without central coordination.

© Pedro Alves 2026

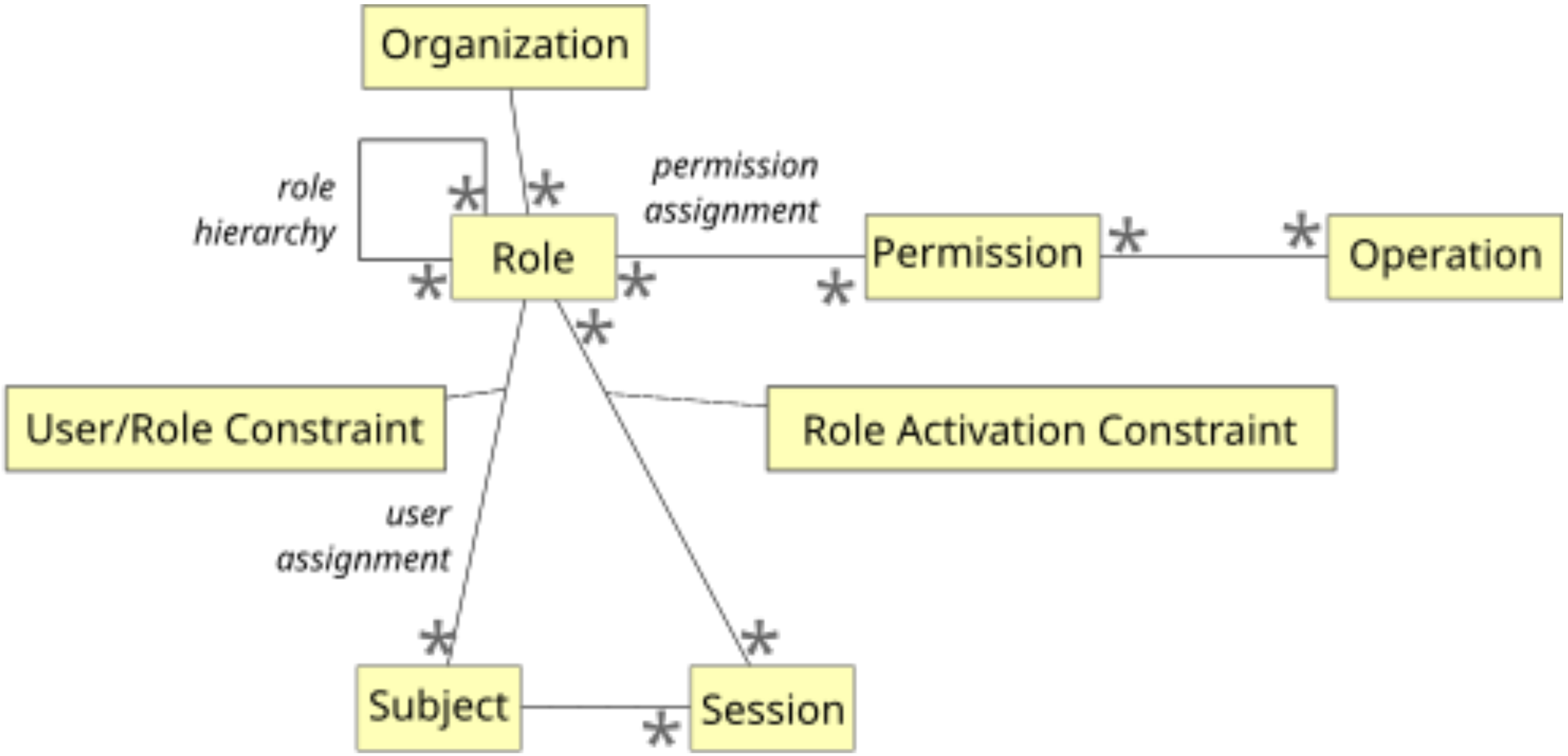
Role-based access control

- Standard approach to authorise users to perform operations in software systems
  - Roles are specified for system related tasks
  - Permissions are assigned to specific roles
  - Users are dynamically assigned roles according to their profile/function



© Pedro Alves 2026

Role-based access control



© Pedro Alves 2026

Model-Based Access Control

Access decisions are based on policies that evaluate multiple factors, such as:

- User attributes (id, department, ownership)

  Example: Only users in the Finance department can view payroll data

- Resource attributes (owner, status)

  Example: Only the creator of the document can edit it

- Context (time, location)

  Example: Sensitive data accessible only from office IP addresses

- Relationships (user <-> resource, user <-> team)

  Example: Managers can view reports submitted by their team members

© Pedro Alves 2026

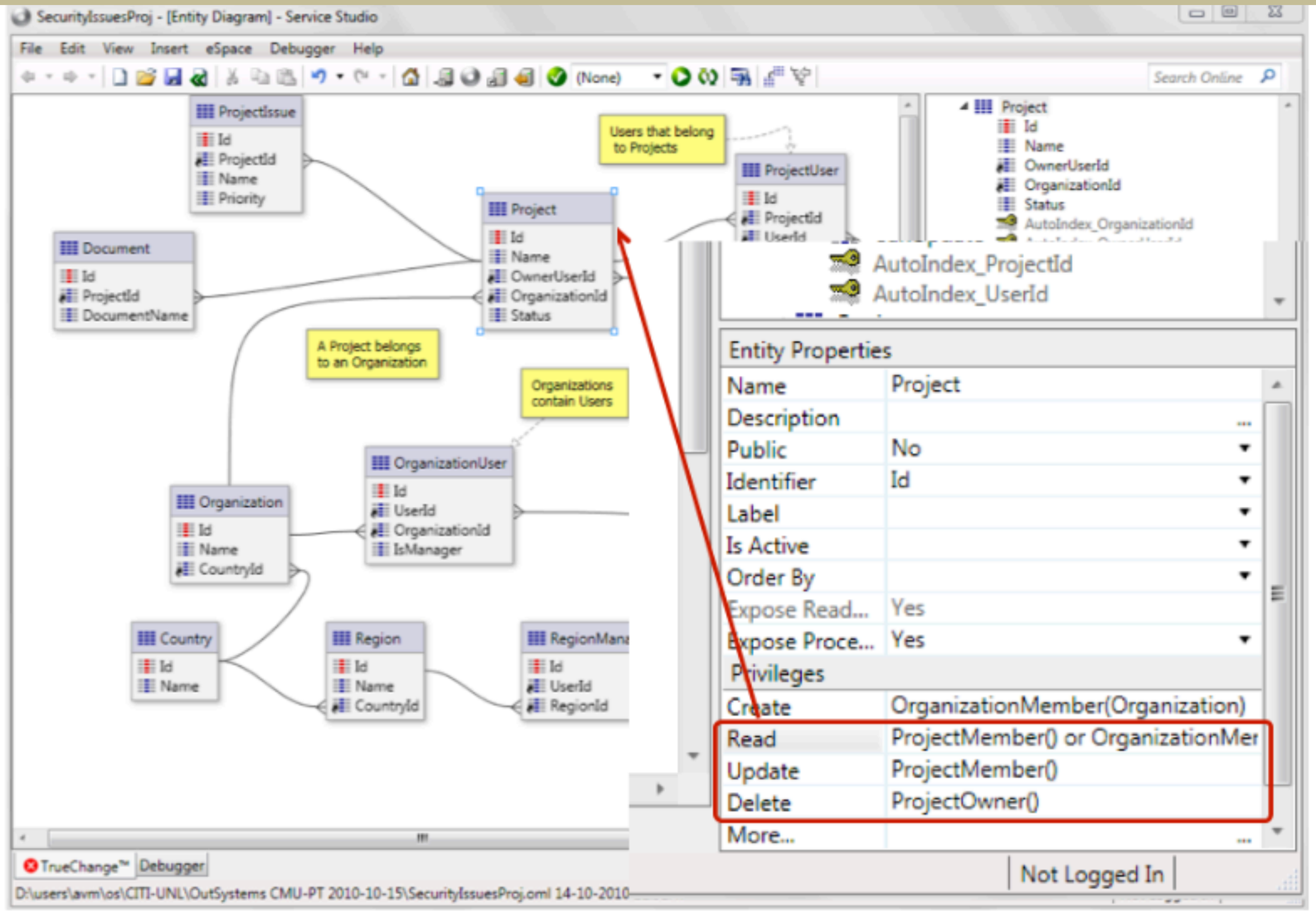
Model-Based Access Control

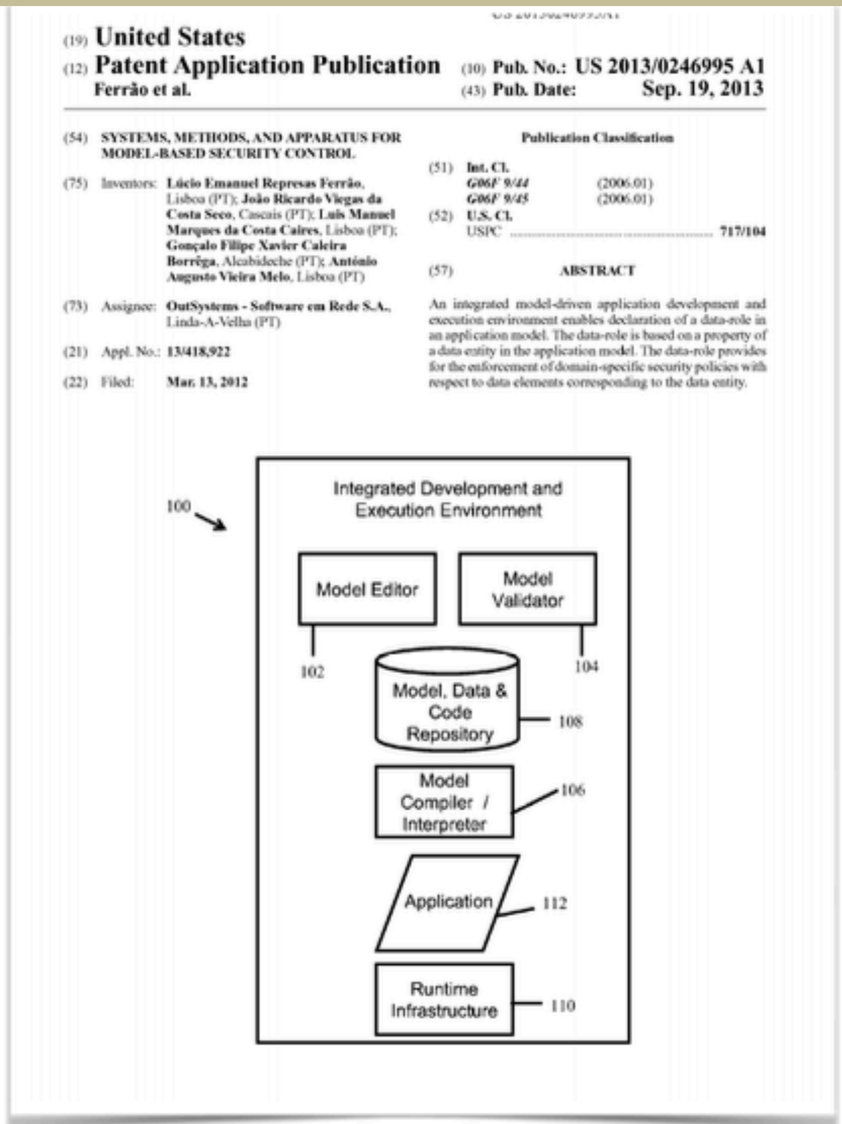
- These rules are many times hidden in query filters and program conditions
- Usually depend on the current state of the application (e.g. `ModeratorOf(…)`) or the target entity (`status == "submitted"`)
- Hard to get right, maintain and modify

© Pedro Alves 2026

Authorization

Model-Based Access Control





(19) United States
(12) Patent Application Publication
Ferrão et al.
(10) Pub. No.: US 2013/0246995 A1
(43) Pub. Date: Sep. 19, 2013

(54) SYSTEMS, METHODS, AND APPARATUS FOR MODEL-BASED SECURITY CONTROL

(75) Inventors: Lúcio Emanuel Represas Ferrão, Lisboa (PT); João Ricardo Viegas da Costa Seco, Cascais (PT); Luís Manuel Marques da Costa Caires, Lisboa (PT); Gonçalo Filipe Xavier Caleira Borrêga, Alcabideche (PT); António Augusto Vieira Melo, Lisboa (PT)

(73) Assignee: OutSystems - Software em Rede S.A., Linda-A-Velha (PT)

(21) Appl. No.: 13/418,922

(22) Filed: Mar. 13, 2012

Publication Classification

(51) Int. Cl.
G06F 9/44 (2006.01)
G06F 9/45 (2006.01)

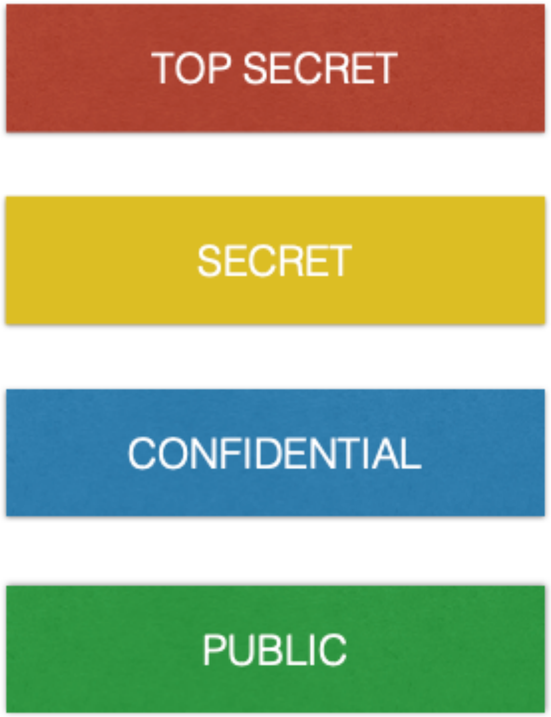
(52) U.S. Cl.
USPC 717/104

(57) ABSTRACT

An integrated model-driven application development and execution environment enables declaration of a data-role in an application model. The data-role is based on a property of a data entity in the application model. The data-role provides for the enforcement of domain-specific security policies with respect to data elements corresponding to the data entity.

Bell-LaPadula model - Confidentiality

- Formalises the U.S. Department of Defense multilevel security policies.
- The security level of a subject is compared to the classification of the object (and the security compartment where it is stored)
- The access control rules state that:
  - A subject cannot read up *(may see information he shouldn't)*
  - A subject cannot write down *(may leak information)*



© Pedro Alves 2026

Bell-LaPadula model - Confidentiality

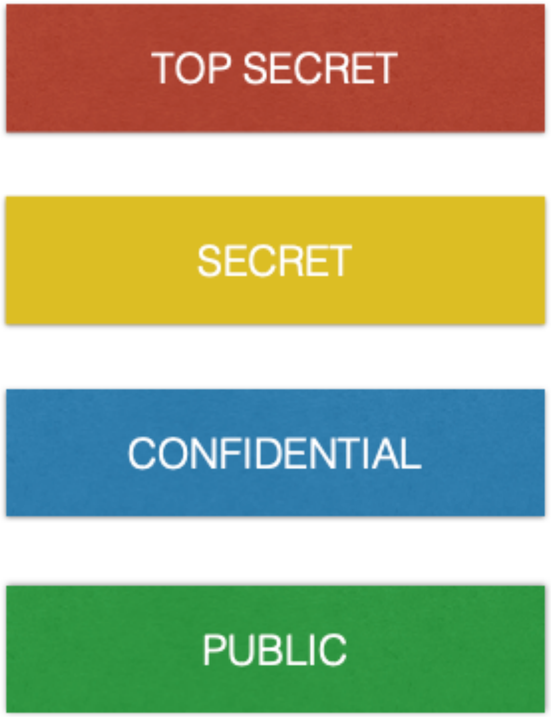
Example

Generals have “Top Secret” clearance

Soldiers have “Confidential” clearance

Soldiers can’t read “Top Secret” mission plans

Generals can’t write into a low-level report



© Pedro Alves 2026

Biba Integrity model - Data Integrity

The Biba model ensures that data remains trustworthy by preventing:

- unauthorized changes,
- improper changes by authorized users,
- and inconsistencies in the system.

- The access control rules state that:
  - A subject cannot read down *(may see low-integrity information that will influence his decisions)*
  - A subject cannot write up *(may contaminate high-integrity data with low-quality information)*

High Integrity

Medium Integrity

Low Integrity

© Pedro Alves 2026

Biba Integrity model - Data Integrity

Example

“Top Secret” mission plans have high integrity

Low-level reports have low integrity

Soldiers can’t write/modify “Top Secret” mission plans

Generals can’t read low-level reports

High Integrity

Medium Integrity

Low Integrity

© Pedro Alves 2026

Authorization

Exercise

	ACLs	Capabilities	RBAC	MBAC
Main entity (policy, token, role, resource)				
How is access checked? (evaluation, lookup, possession)				
Distributed systems fit (excelent, moderate, poor)				
Main benefits				

© Pedro Alves 2026

Authorization

Resolution

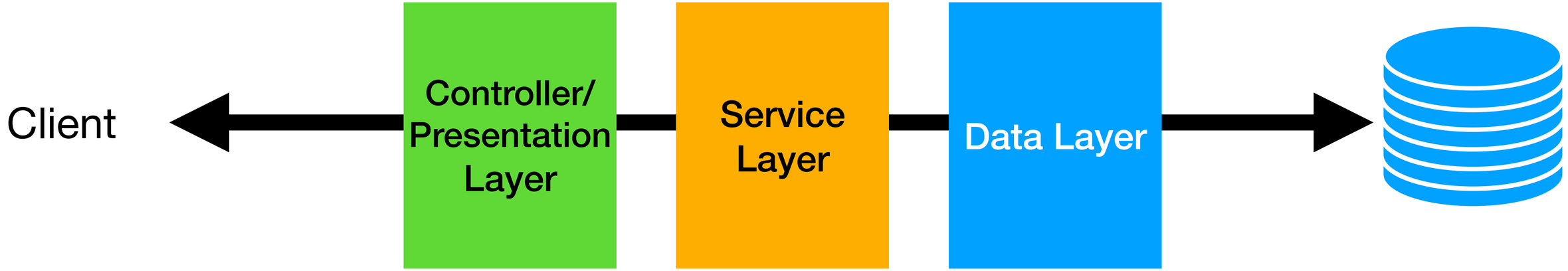
	ACLs	Capabilities	RBAC	MBAC
Main entity (policy, token, role, resource)	resource	token	role	policy
How is access checked? (evaluation, lookup, possession)	lookup	possession	lookup	evaluation
Distributed systems fit (excelent, moderate, poor)	poor (1)	excellent	moderate (2)	moderate (3)
Main benefits	simple to implement	distributed nature	maps well into “real-life” organization roles	handles complex rules

(1) Usually managed by a central authority, can grow a lot, every resource needs to know all users
(2) Still managed by a central authority but resources only need to know roles (much less than users)
(3) Complex rules may include cross-domain attributes

© Pedro Alves 2026

Authorization

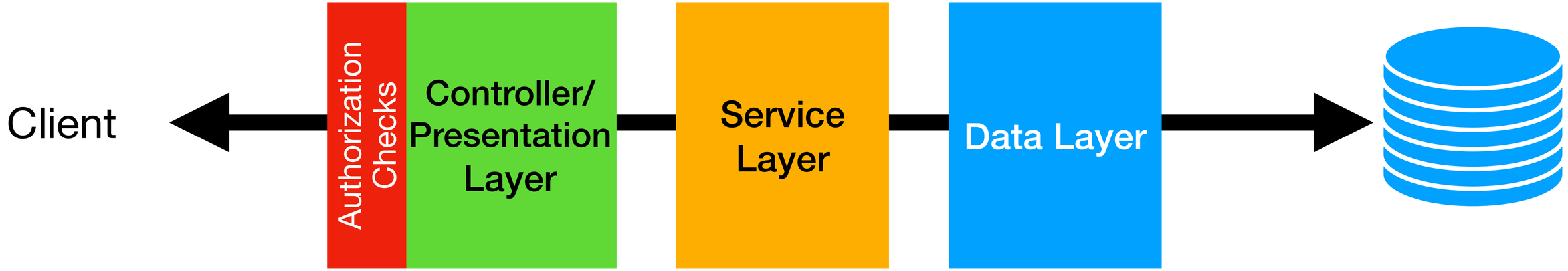
Enforcement location



© Pedro Alves 2026

Authorization

Enforcement location



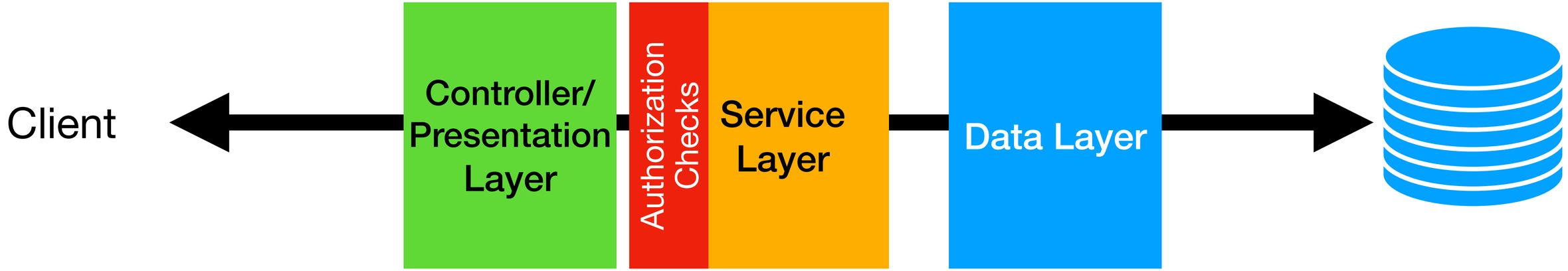
Enforce at the Controller/Presentation Layer:

- ✅ Fast rejection, avoid unnecessary processing
- ✅ Good for coarse-grained access control (e.g., only admins can access this endpoint)
- ❌ Logic gets duplicated across endpoints, not reusable
- ❌ Hard to enforce complex rules

© Pedro Alves 2026

Authorization

Enforcement location



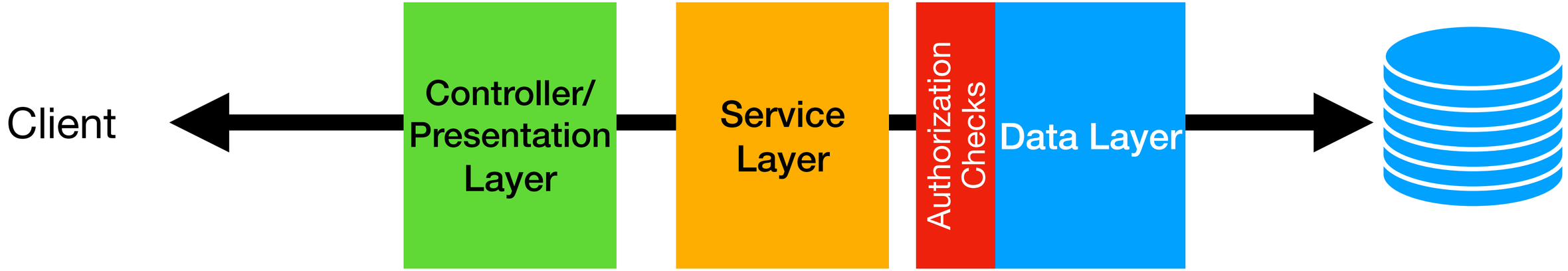
Enforce at the Service Layer:

- ✅ Works across multiple entry points (API, Web, Background Jobs, …)
- ✅ Close to business rules
- ✅ Supports complex logic
- ❌ Bypassable by going direct to the Data Layer

© Pedro Alves 2026

Authorization

Enforcement location



Enforce at the Data Layer:

- ✅ Strongest enforcement - hard to bypass
- ✅ Close to the authorization target
- ❌ Limited expressiveness for complex logic (e.g. team relationships)
- ❌ Potentially slower (each row will have to checked)

© Pedro Alves 2026

Spring Security

Frameworks such as Spring, with Spring Security project promote the reuse of (correct) code, good practices, and great number of base features

- HTTP BASIC authentication headers (an IETF RFC-based standard)
- HTTP Digest authentication headers (an IETF RFC-based standard)
- HTTP X.509 client certificate exchange (an IETF RFC-based standard)
- LDAP (a very common approach to cross-platform authentication needs, especially in large environments)
- Form-based authentication (for simple user interface needs)
- OpenID authentication
- Authentication based on pre-established request headers (such as Computer Associates Siteminder)
- JA-SIG Central Authentication Service (otherwise known as CAS, which is a popular open source single sign-on system)
- Transparent authentication context propagation for Remote Method Invocation (RMI) and HttpInvoker (a Spring remoting protocol)
- Automatic “remember me" authentication (so you can tick a box to avoid re-authentication for a predetermined period of time)
- Anonymous authentication (allowing every unauthenticated call to automatically assume a particular security identity)
- Run-as authentication (which is useful if one call should proceed with a different security identity)
- Java Authentication and Authorization Service (JAAS)
- JEE container autentication (so you can still use Container Managed Authentication if desired)
- Kerberos

© Pedro Alves 2026

Spring Security

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Only adding this, you'll automatically get:

- required authentication for any endpoint
- default user with generated password at startup
- secure password storage
- form-based login and logout flows
- HTTP Basic Authentication
- prevention against several attacks: CSRF, Session Fixation, sniffing, clickjacking, etc..
- ...

© Pedro Alves 2026

Spring Security

Securing a RESTful application with HTTP Basic Authentication

```
@Configuration
@EnableWebSecurity
class SecurityConfig {

    @Bean
    fun passwordEncoder(): PasswordEncoder =
                                BCryptPasswordEncoder()

    @Bean
    fun userDetailsService(encoder: PasswordEncoder) {
       …
    }

    @Bean
    fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
        http
            .csrf { it.disable() }
            .authorizeHttpRequests { auth ->
                auth
                    .requestMatchers(HttpMethod.GET, "/posts", "/posts/**").permitAll()
                    .requestMatchers(HttpMethod.POST, "/posts").hasRole("EDITOR")
                    .requestMatchers(HttpMethod.PUT, "/posts/**").hasRole("EDITOR")
                    .requestMatchers(HttpMethod.DELETE, "/posts/**").hasRole("ADMIN")
                    .anyRequest().authenticated()
            }
            .httpBasic {}

        return http.build()
    }
}
```

without this, security won't work

need to decide how to encript passwords

disable csrf validation for REST requests

authorization rules (role-based)

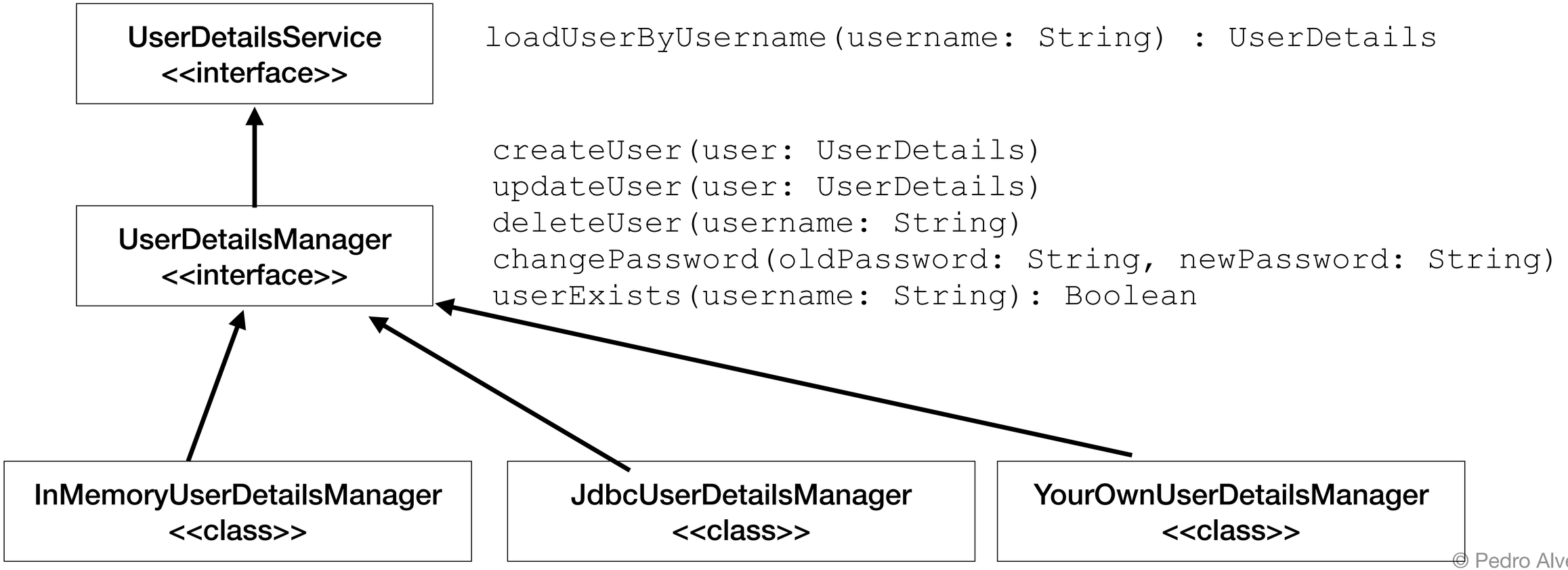
enable Basic Auth

© Pedro Alves 2026

Spring Security

UserDetailsService

UserDetailsService abstracts away how users are retrieved (memory, db, …)



© Pedro Alves 2026

Spring Security

Seeding the application with users

```
@Configuration
@EnableWebSecurity
class SecurityConfig {

    @Bean
    fun passwordEncoder(): PasswordEncoder =
                                BCryptPasswordEncoder()

    @Bean
    fun userDetailsService(encoder: PasswordEncoder) = InMemoryUserDetailsManager(
       User.withUsername("editor")
           .password(encoder.encode("pass"))
           .roles("EDITOR")
           .build(),
       User.withUsername("admin")
           .password(encoder.encode("pass"))
           .roles("ADMIN")
           .build()
    )


    @Bean
    fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
        ...
    }
}
```

in a “real” application you should implement your own UserDetailsManager that retrieves user credentials from a database or from LDAP

© Pedro Alves 2026

Spring Security

Role-based access control

Authorization rules are based on the role and URL patterns

Rules are applied by declaration order (only first match is applied)

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
   http
        ...
        .authorizeHttpRequests { auth ->
           auth
              .requestMatchers(HttpMethod.GET, "/posts", "/posts/**").permitAll()
              .requestMatchers(HttpMethod.POST, "/posts").hasRole("EDITOR")
              .requestMatchers(HttpMethod.PUT, "/posts/**").hasRole("EDITOR")
              .requestMatchers(HttpMethod.DELETE, "/posts/**").hasRole("ADMIN")
              .anyRequest().authenticated()
         }
         ...

   return http.build()
}
```

The last rule should be a catch-all rule that requires authentication for every unmatched URL pattern

© Pedro Alves 2026

Spring Security

Role-based access control

Authorization rules are based on the role and URL patterns

Rules are applied by declaration order

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
   http
        ...
        .authorizeHttpRequests { auth ->
           auth
              .anyRequest().authenticated()
              .requestMatchers(HttpMethod.GET, "/posts", "/posts/**").permitAll()

         }
         ...

   return http.build()
}
```

This rule will never be applied

© Pedro Alves 2026

Spring Security

Role-based access control

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
   http
        ...
        .authorizeHttpRequests { auth ->
           auth
              .requestMatchers(HttpMethod.GET, "/posts", "/posts/**").permitAll()
              .requestMatchers(HttpMethod.POST, "/posts").hasRole("EDITOR")
              .requestMatchers(HttpMethod.PUT, "/posts/**").hasRole("EDITOR")
              .requestMatchers(HttpMethod.DELETE, "/posts/**").hasRole("ADMIN")
              .anyRequest().authenticated()
         }
         ...

   return http.build()
}
```

Faling this returns 403 (Forbidden)

Faling this returns 401 (Unauthorized)

© Pedro Alves 2026

Spring Security

Role-based access control

Authorization rules can also be declared at the method level

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
   http
        ...
        .authorizeHttpRequests { auth ->
           auth
              .requestMatchers(HttpMethod.POST, "/posts").hasRole("EDITOR")
         }
         ...
```

```
@PostMapping("/posts")
@RolesAllowed("EDITOR")
fun create(@RequestBody post: BlogPost): ResponseEntity<Unit> {
    ...
}
```

© Pedro Alves 2026

Spring Security

Role-based access control

Authorization rules can also be declared at the method level

```
@PostMapping("/posts")
@RolesAllowed("EDITOR")
fun create(@RequestBody post: BlogPost): ResponseEntity<Unit> {
    ...
}
```

Also works in methods of “Service” classes

The @RolesAllowed annotation, requires this configuration:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(jsr250Enabled = true)
class SecurityConfig {
```

© Pedro Alves 2026

Spring Security

Once the user is authenticated, a Principal object is created and injected if needed in the controller methods

```
@GetMapping
fun getAll(principal: Principal): List<BlogPost> {
    println("Endpoint is being accessed by: ${principal.name}")
    return posts
}
```

You can also get the Principal like this:

```
@GetMapping
fun getAll(): List<BlogPost> {
    val principal = SecurityContextHolder.getContext().authentication.principal
    println("Endpoint is being accessed by: $principal.name")
    return postService.posts
}
```

© Pedro Alves 2026

```
### Create a new post
POST http://localhost:8080/posts
Authorization: Basic editor pass
Content-Type: application/json

{
  "title": "My New Post",
  "content": "This is the post content.",
  "author": "editor"
}
```

Spring Security

You now have to include the Authorization header in all authentication requests, otherwise you'll get a 401 (Not Authorized)

```
curl -X POST --location "http://localhost:8080/posts" \
    -H "Content-Type: application/json" \
    -d '{
          "title": "My New Post",
          "content": "This is the post content.",
          "author": "editor"
        }' \
   -u editor:pass
```

© Pedro Alves 2026

Exercise

Spot the errors

```
1. @Configuration
2. class SecurityConfig {
3.
4.     fun passwordEncoder(): PasswordEncoder = BCryptPasswordEncoder()
5.
6.     @Bean
7.     fun userDetailsService(encoder: PasswordEncoder): UserDetailsService {
8.         ...
9.     }
10.
11.     @Bean
12.     fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
13.
14.         http
15.             .authorizeHttpRequests { auth ->
16.                 auth
17.                     .requestMatchers(HttpMethod.GET, "/posts/**").permitAll()
18.                     .requestMatchers(HttpMethod.GET, "/posts/private/**").hasRole("EDITOR")
19.                     .requestMatchers(HttpMethod.POST, "/posts").hasRole("EDITOR")
20.                     .requestMatchers(HttpMethod.DELETE, "/posts/**").hasRole("ADMIN")
21.                     .anyRequest().authenticated()
22.             }
23.             .addFilterBefore(PostOwnershipFilter(), // this filter checks the owner of the POST
24.                 UsernamePasswordAuthenticationFilter::class.java)
25.             .httpBasic { }
26.
27.         return http.build()
28.     }
29. }
```

© Pedro Alves 2026

```
1. @Configuration
2. class SecurityConfig {
3.
4.     fun passwordEncoder(): PasswordEncoder = BCryptPasswordEncoder()
5.
6.     @Bean
7.     fun userDetailsService(encoder: PasswordEncoder): UserDetailsService {
8.         ...
9.     }
10.
11.     @Bean
12.     fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
13.
14.         http
15.             .authorizeHttpRequests { auth ->
16.                 auth
17.                     .requestMatchers(HttpMethod.GET, "/posts/**").permitAll()
18.                     .requestMatchers(HttpMethod.GET, "/posts/private/**").hasRole("EDITOR")
19.                     .requestMatchers(HttpMethod.POST, "/posts").hasRole("EDITOR")
20.                     .requestMatchers(HttpMethod.DELETE, "/posts/**").hasRole("ADMIN")
21.                     .anyRequest().authenticated()
22.             }
23.             .addFilterBefore(PostOwnershipFilter(), // this filter checks the owner of the POST
24.                 UsernamePasswordAuthenticationFilter::class.java)
25.             .httpBasic { }
26.
27.         return http.build()
28.     }
29. }
```

missing @EnableWebSecurity

missing @Bean (won't be injected into userDetailsService method)

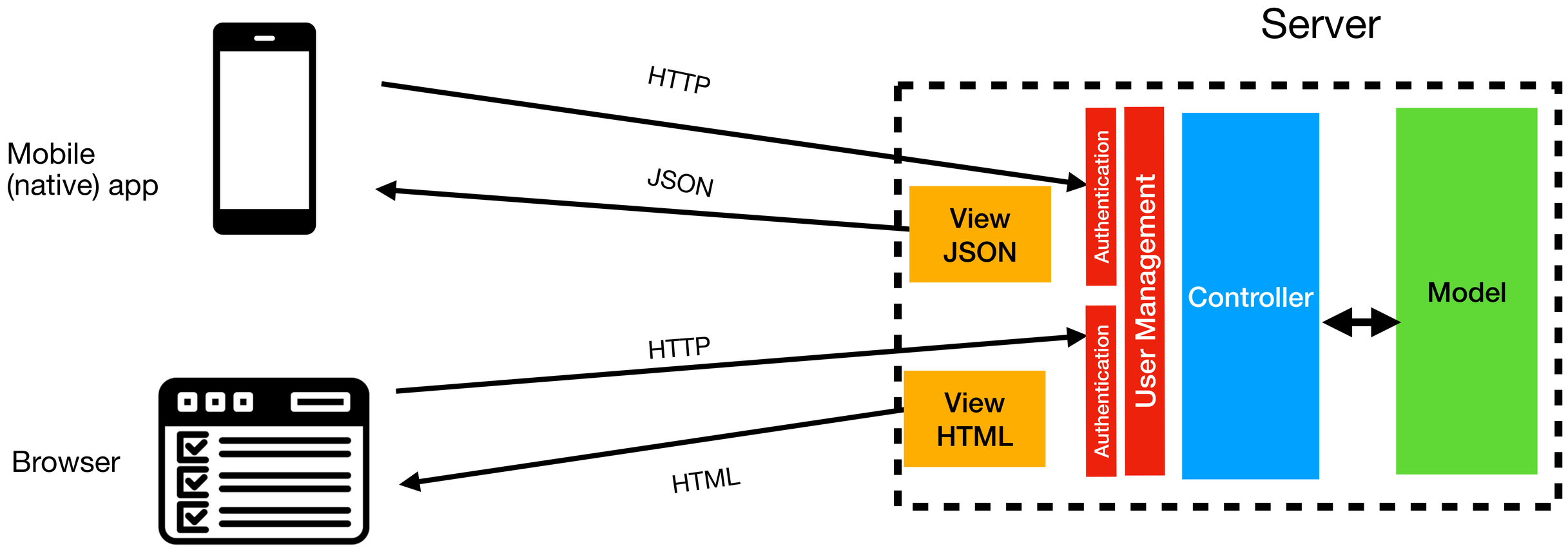
this line is cancelling out the next line, should moved down

missing **.csrf { it.disable() }**

it's impossible to check the owner without first authenticating

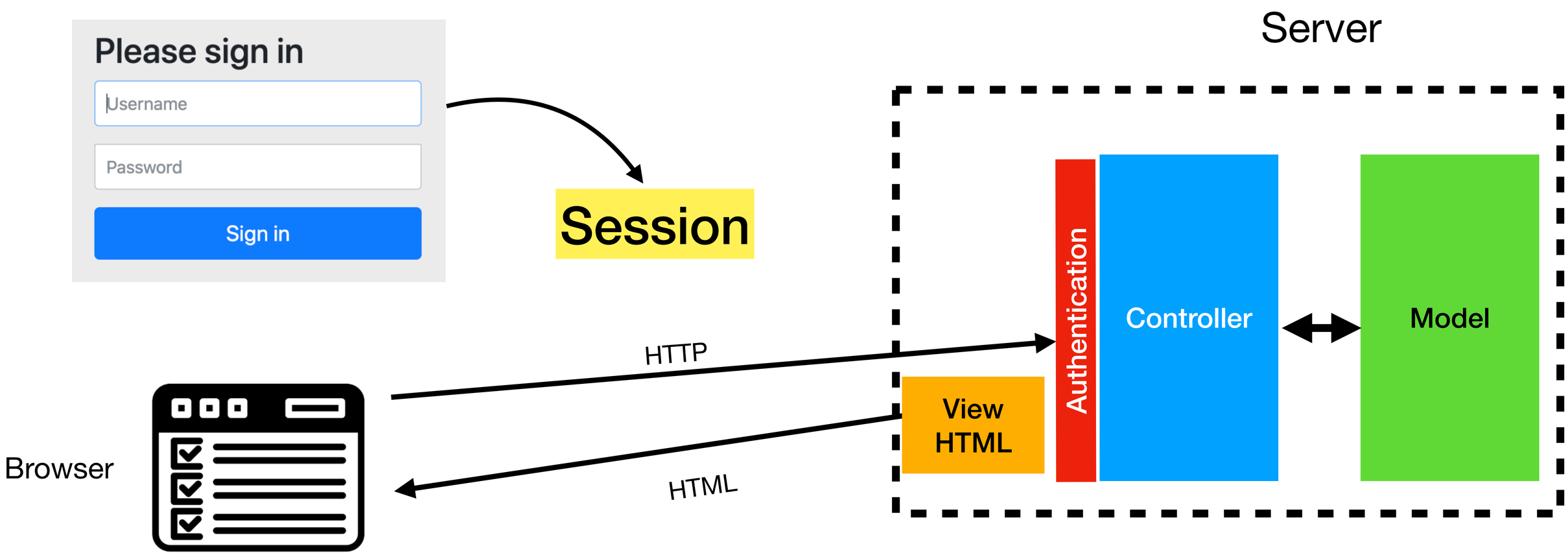
© Pedro Alves 2026

Authentication in Internet Applications



© Pedro Alves 2026

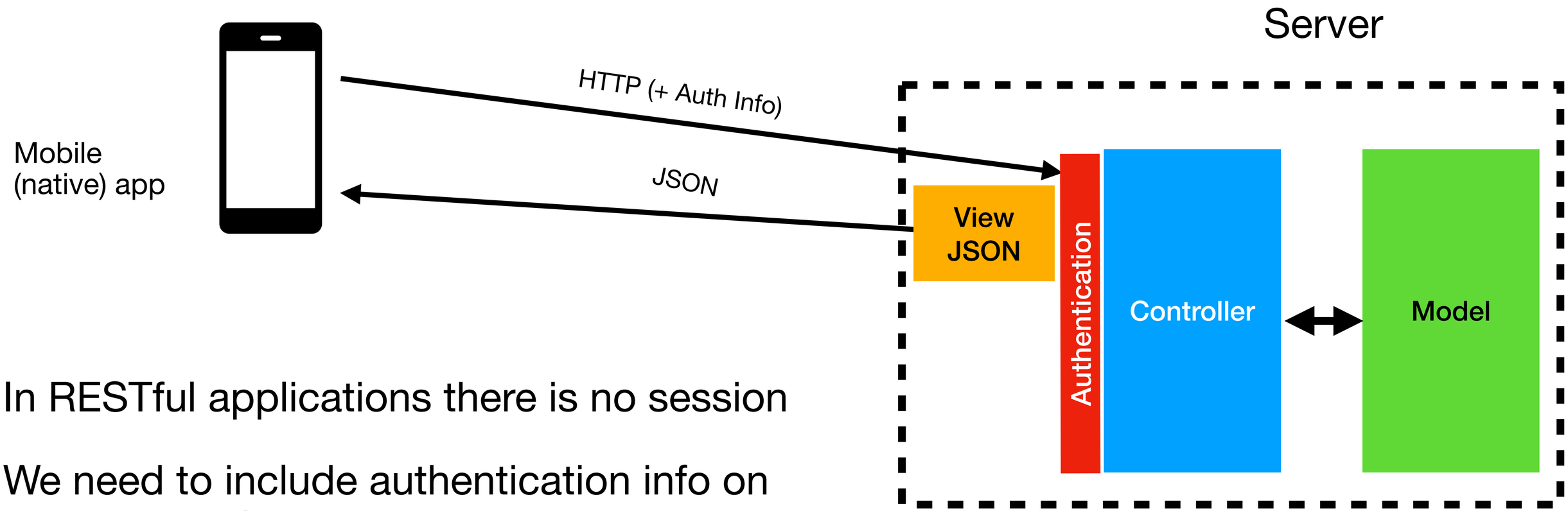
Authentication in Internet Applications



We'll talk about this in the next lesson…

© Pedro Alves 2026

Authentication in Internet Applications



In RESTful applications there is no session

We need to include authentication info on every request

© Pedro Alves 2026

Example

https://www.googleapis.com/customsearch/v1?q=android&key=AIzaSyDfhpqYkH0WGUzDgd3HdSjWmM-Zs8968EA


```
{
 "kind": "customsearch#search",
 "url": {
  "type": "application/json"
 },
 "items": [
  {
   "title": "Android",
   "link": "https://www.android.com/",
   "displayLink": "www.android.com",
   "snippet": "See what's new with Android - from phones to watches and more. Visit the official \nsite to
explore and learn.”
  },
  {
   "title": "Android (operating system) - Wikipedia",
   "link": "https://en.wikipedia.org/wiki/Android_(operating_system)",
   "displayLink": "en.wikipedia.org",
   …
  }
  ...
```


© Pedro Alves 2026

Example

https://www.googleapis.com/customsearch/v1?q=android&key=AIzaSyDfhpqYkH0WGUzDgd3HdSjWmM-Zs8968EA

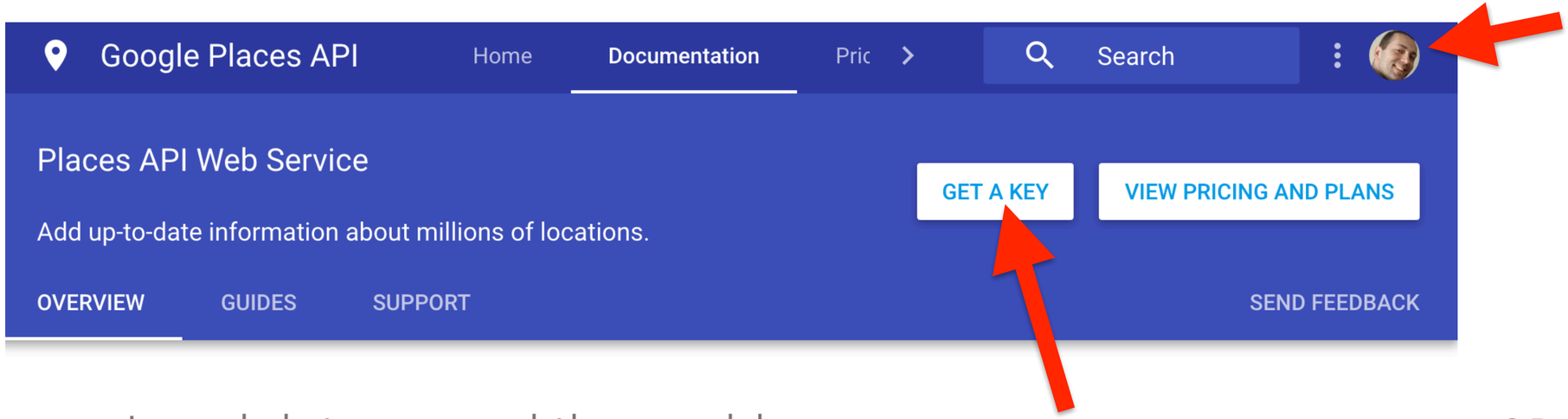
What is this “key” parameter?

© Pedro Alves 2026

API Token

Most public APIs (authenticated or not) require an API token / API key

- This token authenticates the client application, not the user
- It is usually obtained upon login by the party responsible for the client application (e.g., the company developing a mobile app)



Local data around the world

© Pedro Alves 2026

API Token

Most public APIs (authenticated or not) require an API token / API key

- This token authenticates the client application, not the user
- It is usually obtained upon login by the party responsible for the client application (e.g., the company developing a mobile app)
- It helps prevent API abuse and attacks, and can support usage-based pricing models (e.g., free up to 1,000 requests per day)
- The token can be passed through an HTTP Header (preferable, more REST) or a query parameter

© Pedro Alves 2026

API Token - Example

Example from data.gov

API Key Usage

After signing up, you'll be given your own, unique API key.

Ways to Pass Your API Key

HTTP Header

Pass the API key into the `X-Api-Key` header:

```
curl -H 'X-Api-Key: DEMO_KEY' 'https://developer.nrel.gov/api/alt-fuel-stations/v1.json?limit=1'
```

GET Query Parameter

In some cases, the API may support passing the API key into the `api_key` GET query string parameter:

```
curl 'https://developer.nrel.gov/api/alt-fuel-stations/v1.json?limit=1&api_key=YOUR_KEY_HERE'
```

https://api.data.gov/docs/developer-manual/

© Pedro Alves 2026

OpenAPI - API Token

OpenAPI includes a section to describe how the API token should be transmitted

```yaml
openapi: 3.0.4
---
# 1) Define the key name and location
components:
  securitySchemes:
    ApiKeyAuth: # arbitrary name for the security scheme
      type: apiKey
      in: header # can be "header", "query" or "cookie"
      name: X-API-KEY # name of the header, query parameter or cookie

# 2) Apply the API key globally to all operations
security:
  - ApiKeyAuth: [] # use the same name as under securitySchemes
```

https://swagger.io/docs/specification/v3_0/authentication/api-keys/

© Pedro Alves 2026

OpenAPI - API Token

Spring generates this part of the OpenAPI spec based on the SecuritySchemes annotation

```
@Configuration
@EnableWebSecurity
@SecuritySchemes(
    SecurityScheme(
        name = "apiToken",
        type = SecuritySchemeType.APIKEY,
        `in` = SecuritySchemeIn.HEADER,
        paramName = "X-Api-Token",
    ),
)
class SecurityConfig {
   ...
}
```

© Pedro Alves 2026

API Token

API Token validation errors should result in standard HTTP error codes:

- **401** (Unauthorized) - Token is absent, invalid, or expired
- **429** (Too many requests) - Rate limit exceeded

© Pedro Alves 2026

User Authentication

Besides authenticating the client application, some APIs also require user authentication

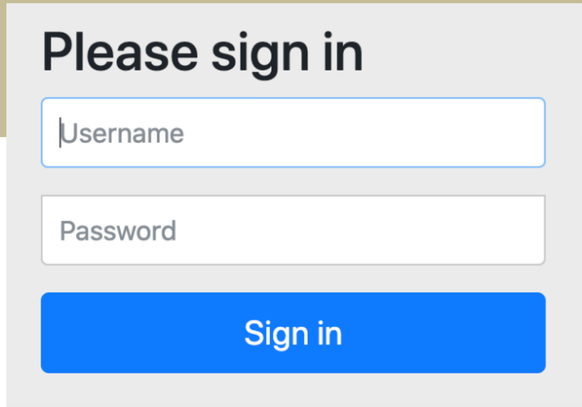
Example:

```
GET https://api.spotify.com/v1/me/playlists
```

Get my (user) playlists

© Pedro Alves 2026

User Authentication



HTTP Basic Auth

Uses the same username/pass of form-based login but sent in a HTTP Header

```
curl -u username:password https://api.example.com/resource
```

or

```
### Example
GET https://api.example.com/resource
Authorization: Basic username password
```

Problem:

The password is sent on every request

```
Authorization: Basic <base64-encoded-credentials>
```

© Pedro Alves 2026

User Authentication

Tokens

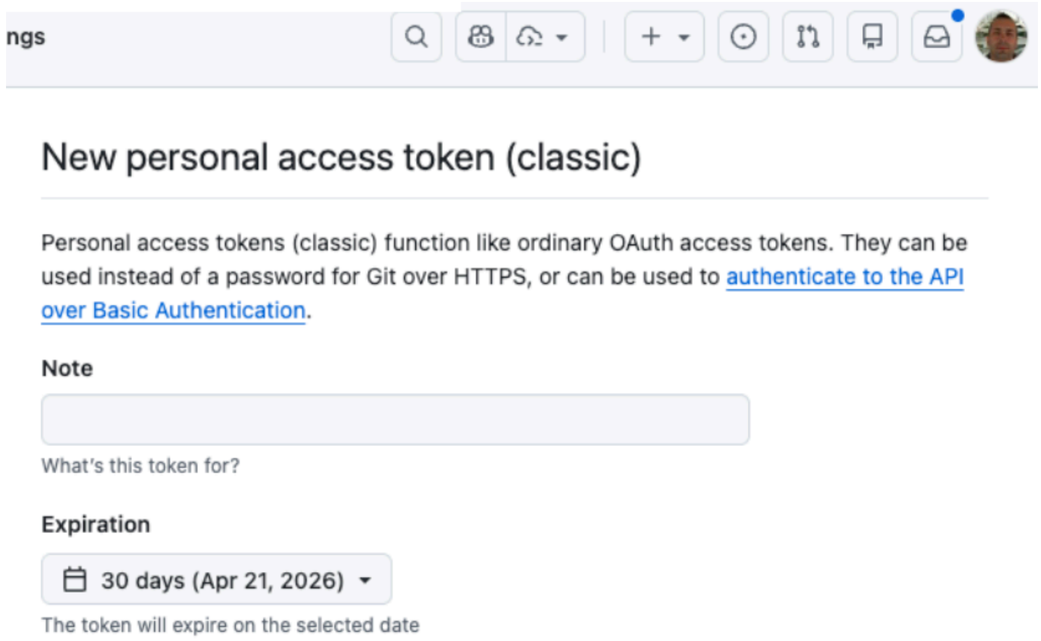
- Authenticate users without exposing passwords
- Time-limited validity (expiration)
- Easily revocable
- Scoped access (e.g., read-only)

© Pedro Alves 2026

User Authentication

User access token obtained upon login

```
Authorization: token <token>
```



https://github.com/settings/tokens/new

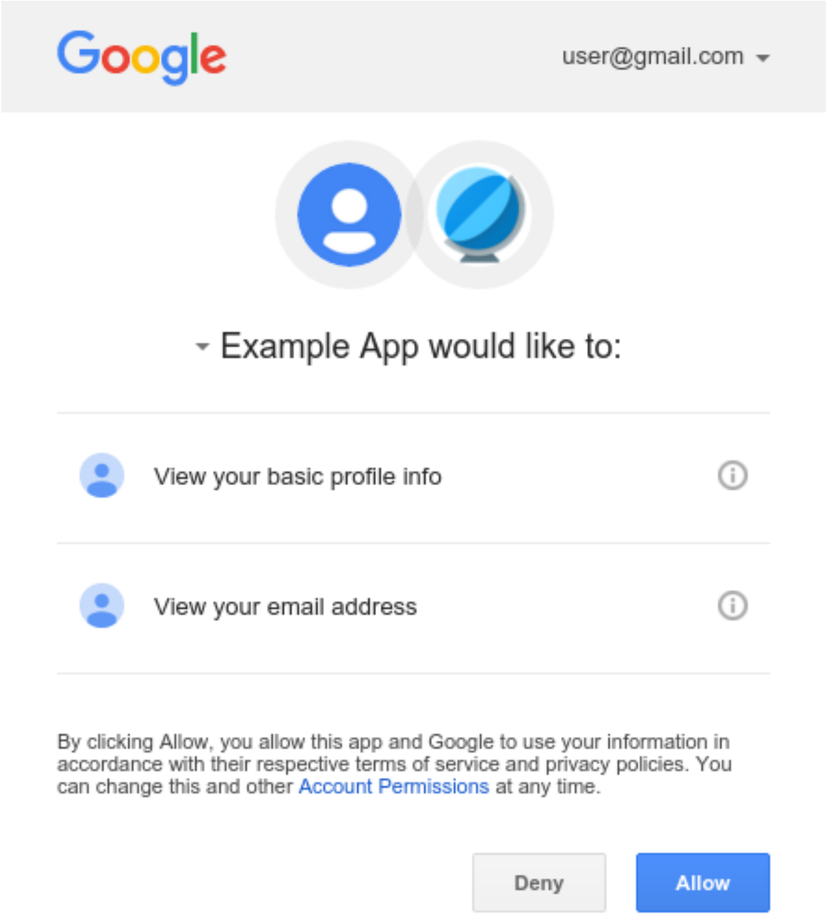
© Pedro Alves 2026

User Authentication

OAuth 2.0

Delegates authentication to a third-party provider (e.g., Google, GitHub) and issues an access token for the user without sharing their credentials

```
Authorization: Bearer ACCESS_TOKEN
```



© Pedro Alves 2026

User Authentication

JWT (JSON Web Token)

Token includes user info + claims, in JSON format.
Usually used in conjunction with OAuth

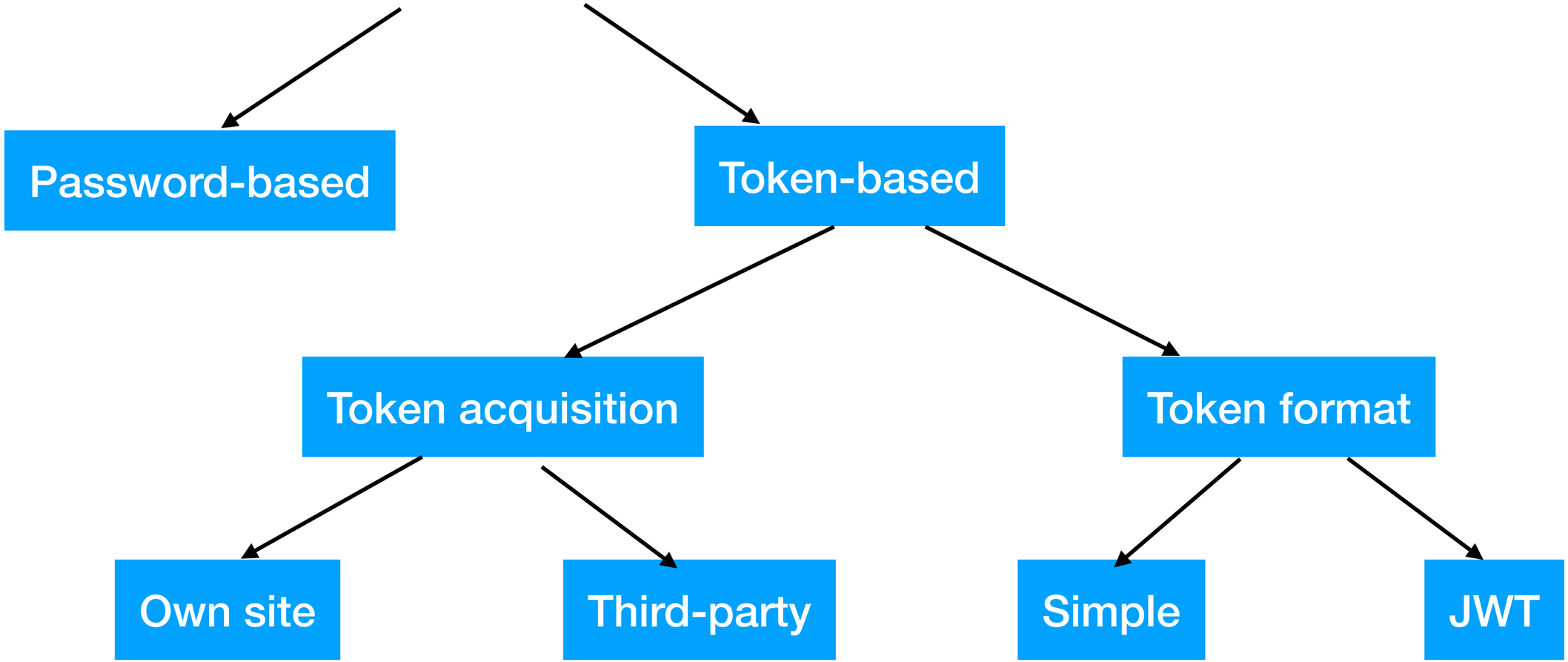
```
Authorization: Bearer JWT_TOKEN
```

```
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "sub": "1234567890",
    "name": "John Doe",
    "role": "admin",
    "scope": "read:orders write:orders",
    "iat": 1516239022
  },
  "signature": "SflKxwRJSMeKKF2QT4fw..."
}
```

encoded

© Pedro Alves 2026

User Authentication



© Pedro Alves 2026

User Authentication - OpenAPI

```yaml
components:
  securitySchemes:
    BasicAuth:
      type: http
      scheme: basic

paths:
  /public:
    get:
      summary: Public endpoint (no auth)
      responses:
        '200':
          description: OK

  /private:
    get:
      summary: Protected endpoint (Basic Auth required)
      security:
        - BasicAuth: []
      responses:
        '200':
          description: OK
        '401':
          description: Unauthorized
```

Example for Basic Auth, it supports other authentication schemas

© Pedro Alves 2026

User Authentication - OpenAPI

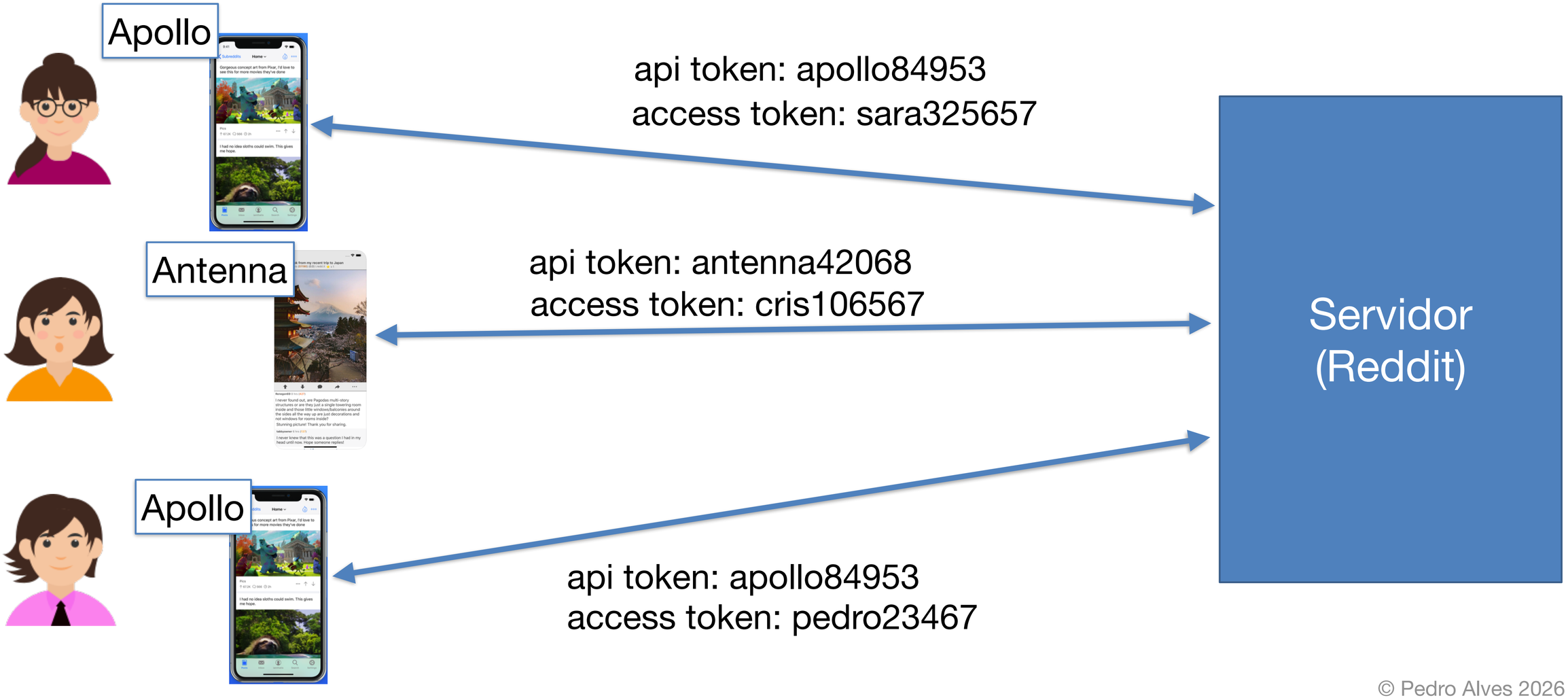
```yaml
components:
  securitySchemes:
    BasicAuth:
      type: http
      scheme: basic

paths:
  /public:
    get:
      summary: Public endpoint (no auth)
      responses:
        '200':
          description: OK

  /private:
    get:
      summary: Protected endpoint
      security:
        - BasicAuth: []
      responses:
        '200':
          description: OK
        '401':
          description: Unauthorized
```

Example for Basic Auth, it supports other authentication schemas as well

```java
@Configuration
@EnableWebSecurity
@SecuritySchemes(
    SecurityScheme(name = "basicAuth",
                   type = SecuritySchemeType.HTTP,
                   scheme = "basic"),
  )
class SecurityConfig {
 ...
}
```

© Pedro Alves 2026

API Token vs Access Token

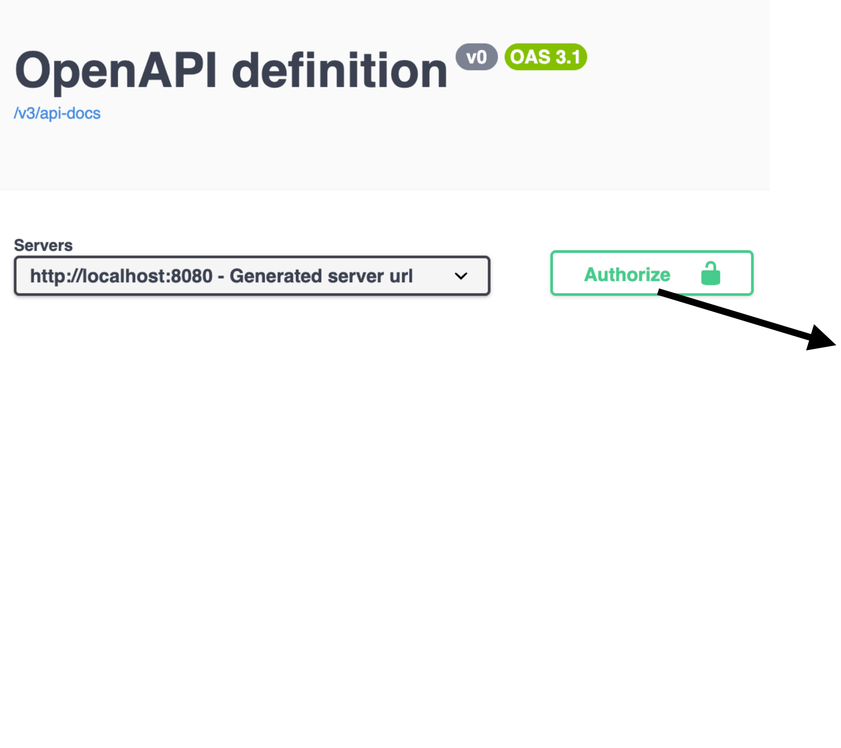


© Pedro Alves 2026

Security
OpenAPI

Security schemes should be included the OpenAPI Spec:

```
"securitySchemes": {
    "apiToken": {
        "type": "apiKey",
        "name": "X-Api-Token",
        "in": "header"
    },
    "basicAuth": {
        "type": "http",
        "scheme": "basic"
    }
}
```



© Pedro Alves 2026

Security

OpenAPI

In Spring, there is the `@SecuritySchemes` annotation

```
@SecuritySchemes(
    SecurityScheme(
        name = "basicAuth",
        type = SecuritySchemeType.HTTP,
        scheme = “basic”,
    ),
    SecurityScheme(
        name = "apiToken",
        type = SecuritySchemeType.APIKEY,
        `in` = SecuritySchemeIn.HEADER,
        paramName = "X-Api-Token",
    ),
)
class SecurityConfig {
  ...
}
```

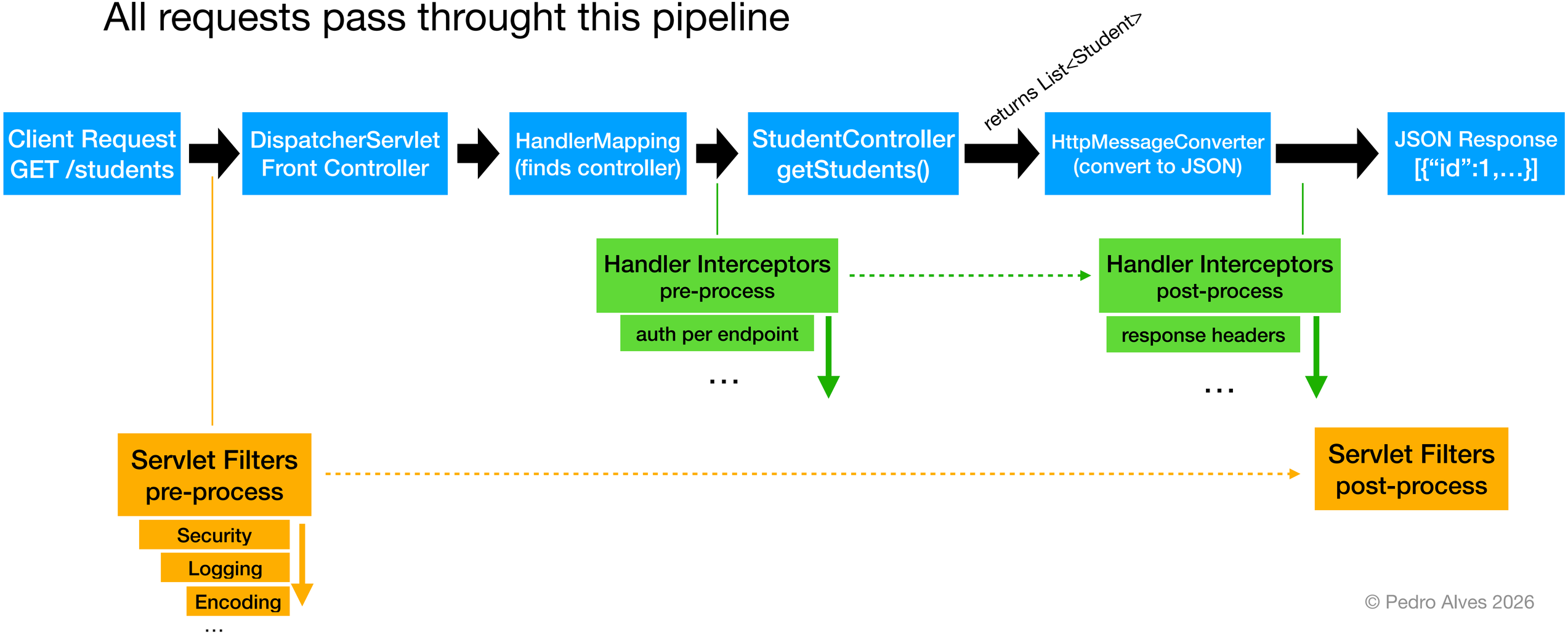
© Pedro Alves 2026

REST server implementation

Spring MVC

Revisions from week 2

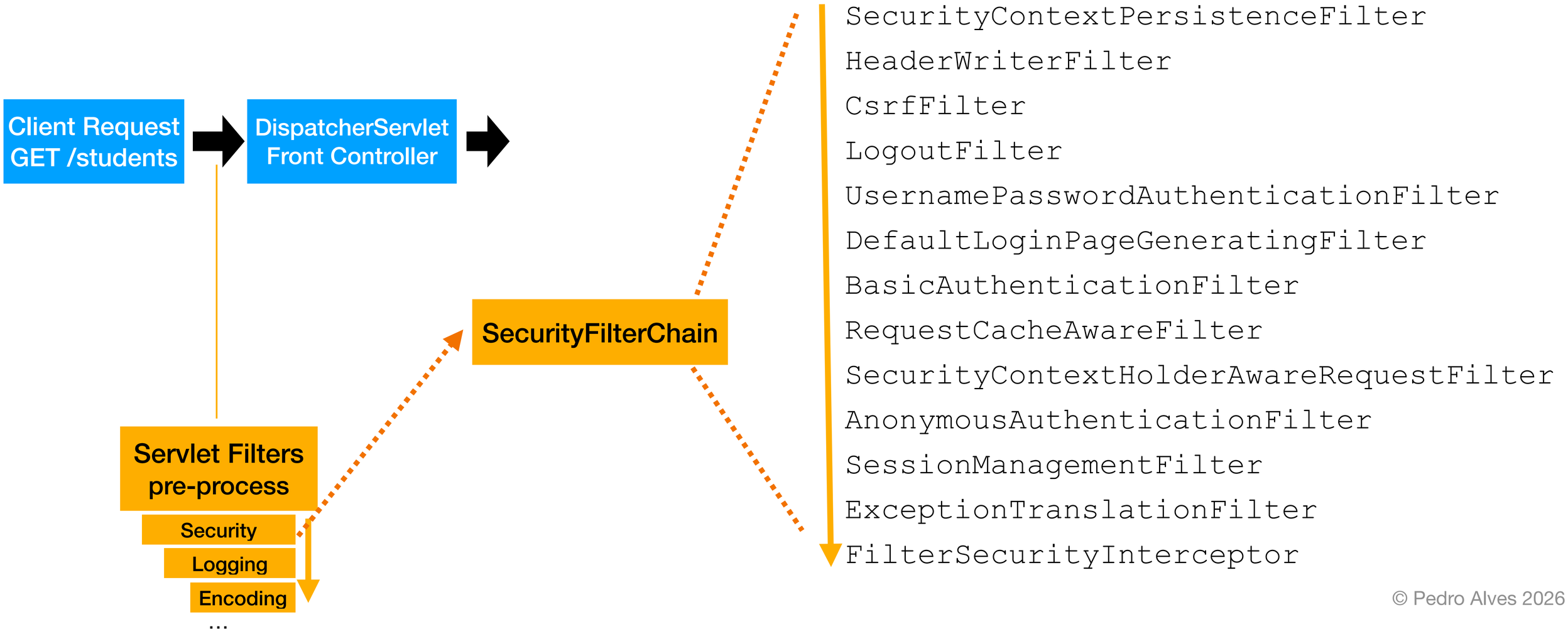
All requests pass throught this pipeline



© Pedro Alves 2026

Spring

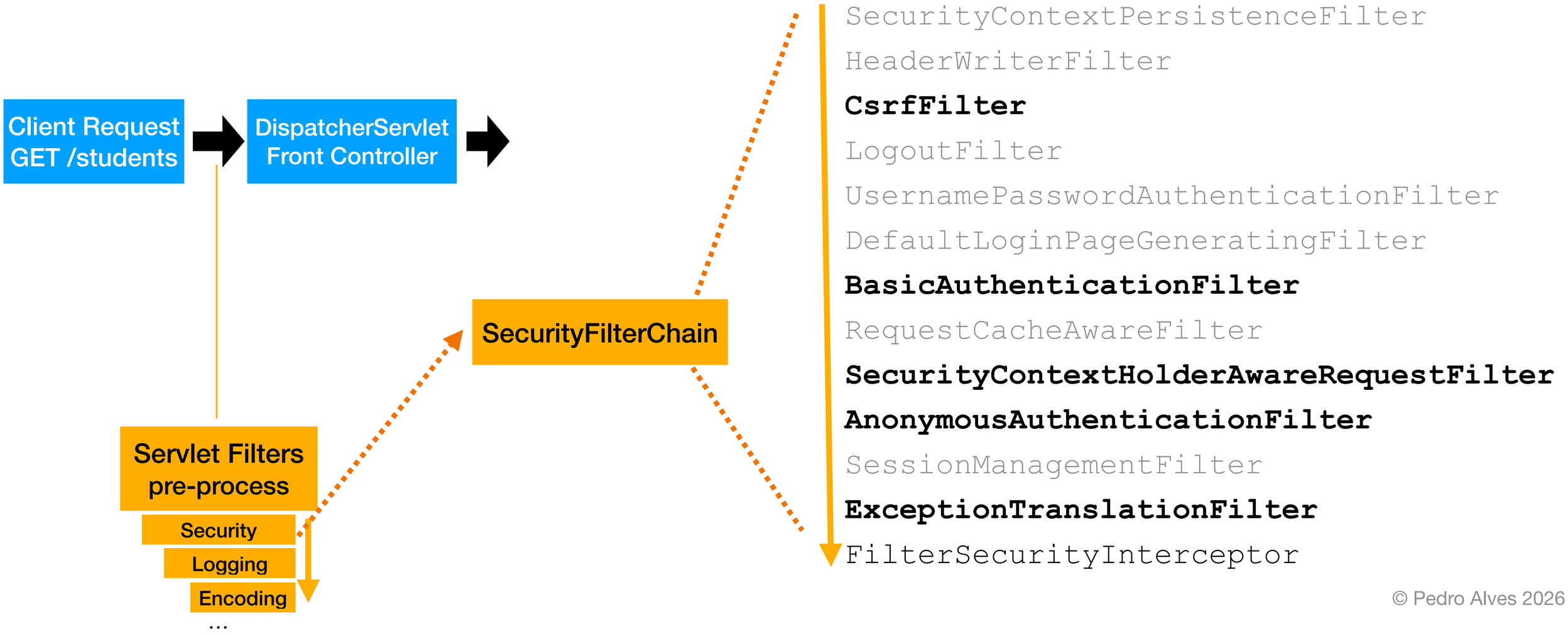
SecurityFilterChain



© Pedro Alves 2026

Spring

SecurityFilterChain



© Pedro Alves 2026

Spring

SecurityFilterChain

- SecurityContextHolderFilter
- HeaderWriterFilter
- **CsrfFilter** → Validates csrf token in POST,PUT,DELETE
- LogoutFilter
- UsernamePasswordAuthenticationFilter
- DefaultLoginPageGeneratingFilter
- **BasicAuthenticationFilter** → Handles HTTP Basic Auth
- RequestCacheAwareFilter
- **SecurityContextHolderAwareRequestFilter** → Wraps HttpServletRequest with a Principal
- **AnonymousAuthenticationFilter** → If not authenticated, create anonymous Principal
- SessionManagementFilter
- **ExceptionTranslationFilter** → Catches security exceptions (Authentication failures → 401, Access denied → 403)
- **FilterSecurityInterceptor** → Applies authorization rules

© Pedro Alves 2026

Spring

SecurityFilterChain

You can configure/tweak the default chain

SecurityContextHolderFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor

```
@Bean
fun securityFilterChain(http: HttpSecurity):
                                SecurityFilterChain {
   http
     .csrf { it.disable() }
     .authorizeHttpRequests { auth ->
          auth.anyRequest().authenticated()
     }
     .httpBasic {}
     .exceptionHandling { exceptions ->
        exceptions.accessDeniedHandler {
          _, response, _ ->
           response.status = 403
           response.contentType = …
           response.writer.write(…)
     }

   return http.build()
}
```

© Pedro Alves 2026

Spring

SecurityFilterChain

SecurityContextHolderFilter

HeaderWriterFilter

CsrfFilter

LogoutFilter

ApiTokenFilter

UsernamePasswordAuthenticationFilter

DefaultLoginPageGeneratingFilter

BasicAuthenticationFilter

RequestCacheAwareFilter

SecurityContextHolderAwareRequestFilter

AnonymousAuthenticationFilter

SessionManagementFilter

ExceptionTranslationFilter

FilterSecurityInterceptor

You can add your own filters

```
@Bean
fun securityFilterChain(http: HttpSecurity):
                                SecurityFilterChain {
   http
      .csrf { it.disable() }
      .addFilterBefore(ApiTokenFilter(),
         UsernamePasswordAuthenticationFilter::class.java)
      .authorizeHttpRequests { auth ->
            auth.anyRequest().authenticated()
      }
      .httpBasic {}
      .exceptionHandling { exceptions ->
         …

   return http.build()
}
```

© Pedro Alves 2026

Spring

SecurityFilterChain

Custom authentication filters:

- Extend `OncePerRequestFilter`
- Usually inserted before `UsernamePasswordAuthenticationFilter`
- If authentication is valid:
  - call the next filter in the chain (Decorated Intercepting Filter Pattern)
- If authentication is invalid:
  - return (breaks the chain)

© Pedro Alves 2026

Spring

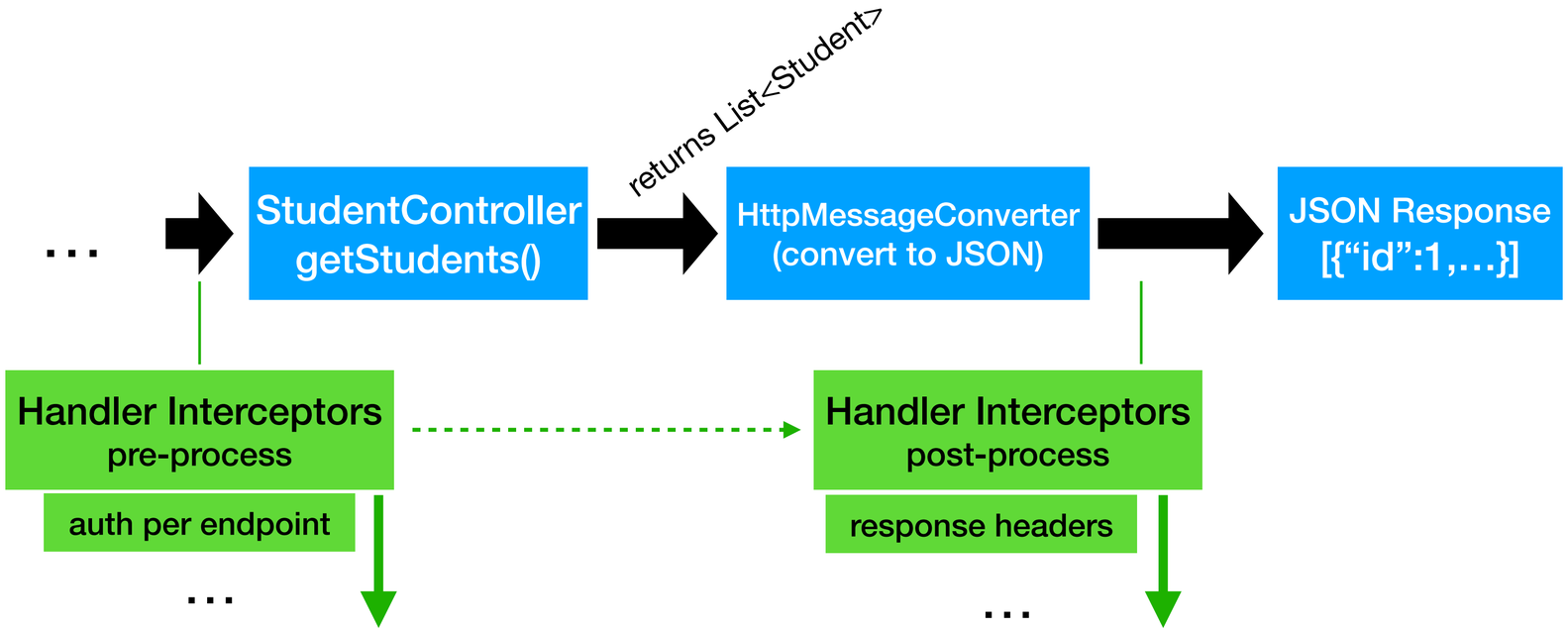
SecurityFilterChain

```
class SomeFilter : OncePerRequestFilter() {

    override fun doFilterInternal(
        request: HttpServletRequest,
        response: HttpServletResponse,
        filterChain: FilterChain,
    ) {

        // get auth info from request and validate...

        if (!isValidAuth) {
            response.status = HttpServletResponse.SC_UNAUTHORIZED
            response.contentType = "application/json"
            response.writer.write(“”“{"error":"UNAUTHORIZED","message":"..."}”””)
            return
        }

        filterChain.doFilter(request, response)
    }
}
```

© Pedro Alves 2026

Spring

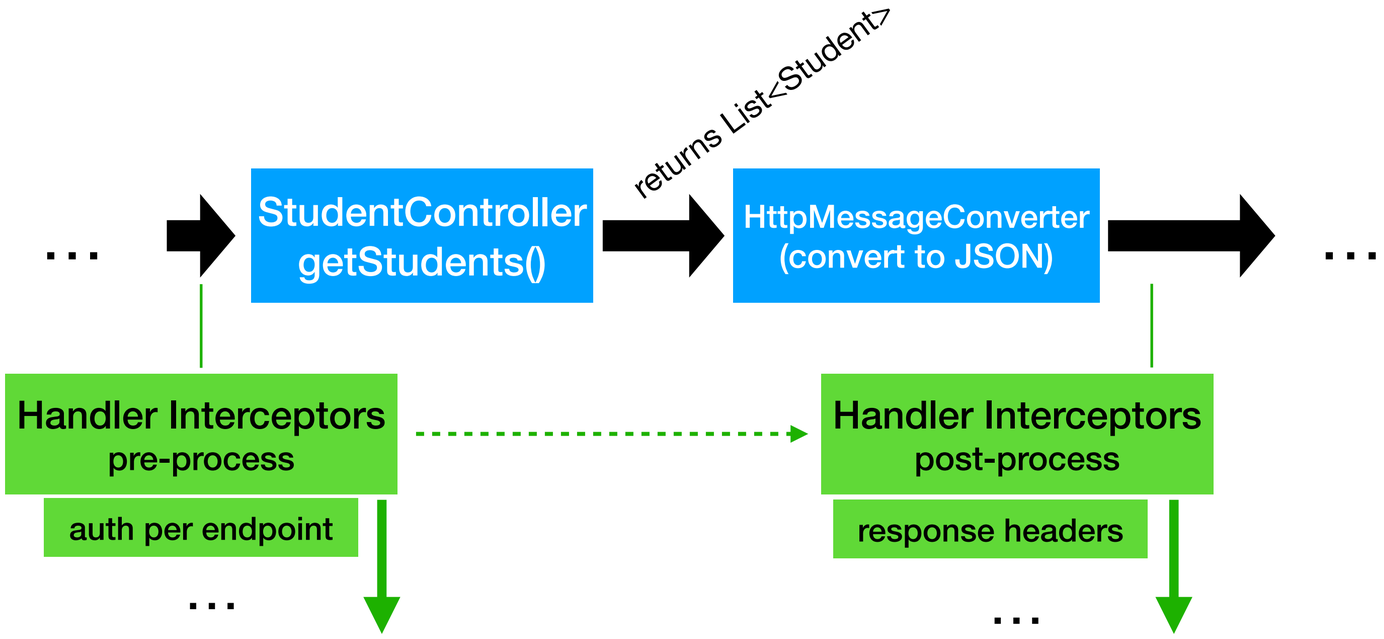
Handler Interceptor

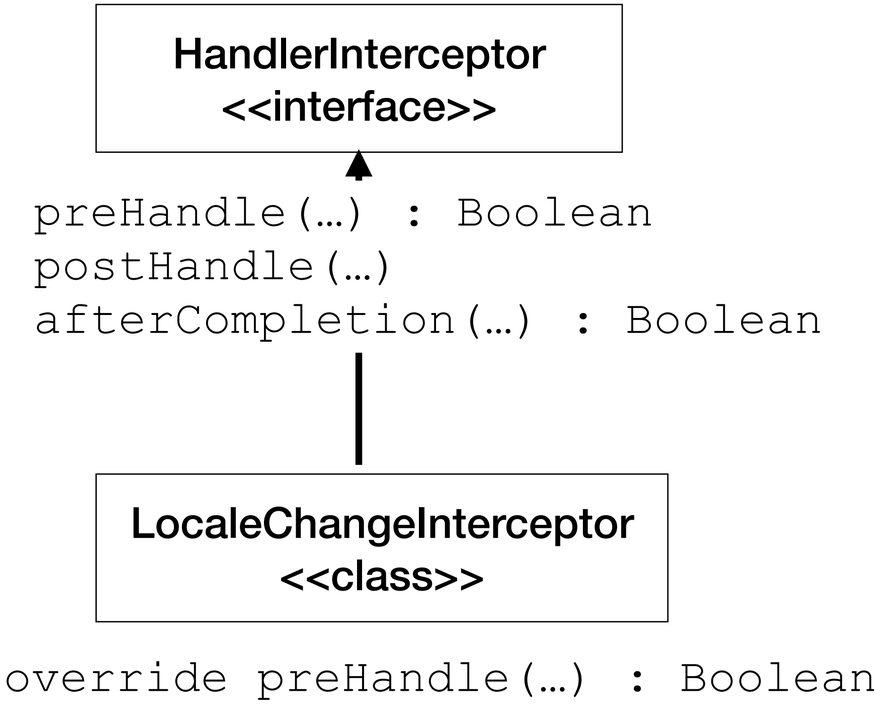


© Pedro Alves 2026

Spring

Handler Interceptor





© Pedro Alves 2026

Spring

Handler Interceptor

Example: Logs all HTTP requests

```
@Component
class LoggingInterceptor(private val registry: ApiTokenRegistry) : HandlerInterceptor {

    private val log = LoggerFactory.getLogger(LoggingInterceptor::class.java)

    override fun afterCompletion(
        request: HttpServletRequest,
        response: HttpServletResponse,
        handler: Any,
        ex: Exception?,
    ) {
        val appName = registry.tokenToApp[request.getHeader("X-Api-Token")] ?: "unknown"
        val principal = request.userPrincipal?.name ?: "anonymous"
        log.info("[{}] [{}] {} {} [{}]", appName, principal, request.method, request.requestURI, response.status)
    }
}
```

Notice that this implements the non-decorated intercepting filter pattern

© Pedro Alves 2026

Spring

Handler Interceptor

To add an interceptor:

```
@Configuration
class MvcConfig(private val loggingInterceptor: LoggingInterceptor) : WebMvcConfigurer {

    override fun addInterceptors(registry: InterceptorRegistry) {
        registry.addInterceptor(loggingInterceptor)
    }
}
```

© Pedro Alves 2026

Spring

Model-based access control

Example: Only the original author of the blog post can replace it

```
@PutMapping("/posts/{id}")
fun update(@PathVariable id: Long, @RequestBody updated: BlogPost,
           principal: Principal):
      ResponseEntity<BlogPost>
 {
    val post = postService.findById(id)
    ?: throw ResponseStatusException(HttpStatus.NOT_FOUND,
            “Post $id not found")

    if (post.author != principal.name)
        throw ResponseStatusException(HttpStatus.FORBIDDEN,
                                      "Only the author can replace this post")
    ...
}
```

© Pedro Alves 2026

Spring

Model-based access control

Example: Only the original author of the blog post can replace it

```
@PutMapping("/posts/{id}")
fun update(@PathVariable id: Long, @RequestBody updated: BlogPost,
           principal: Principal):
      ResponseEntity<BlogPost>
 {
    val post = postService.findById(id)
    ?: throw ResponseStatusException(HttpStatus.NOT_FOUND,
            “Post $id not found")

    if (post.author != principal.name)
        throw ResponseStatusException(HttpStatus.FORBIDDEN,
                                      "Only the author can replace this post")
    ...
}
```

Authorization rules should be declared, not programmed

© Pedro Alves 2026

Spring

Model-based access control

The `@PreAuthorize` annotation evaluates a SPEL expression which can reference Spring components, principal information and parameters

```
Service
class PostService {
    ...
    fun isAuthor(id: Long, name: String): Boolean = ...
}
```

```
@PutMapping("/posts/{id}")
@PreAuthorize("@postService.isAuthor(#id, principal.name)")
fun update(@PathVariable id: Long, @RequestBody updated: BlogPost):
      ResponseEntity<BlogPost> {
    val post = postService.findById(id)
        ?: throw ResponseStatusException(HttpStatus.NOT_FOUND, "Post $id not found")
    ...
}
```

© Pedro Alves 2026

Spring

Model-based access control

Encoding authorization rules into @PreAuthorized annotations

- ✅ Doesn’t pollute the controller/business logic with authorization checks
- ✅ Declares the rules instead of programming them
- ❌ No compile-time checking (SPEL is evaluated in runtime)
- ❌ Possible duplication of rules (if the same rule is applied in multiple places)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
class SecurityConfig {
```

Add this annotation to use @PreAuthorize

© Pedro Alves 2026

Spring

SpEL

Spring Expression Language (SpEL) lets you write fine-grained access rules using expressions instead of hardcoding logic into your code

Usually used with @PreAuthorize annotations

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }
```

```
@PreAuthorize("#id == authentication.principal.id")
public User getUser(Long id) { ... }
```

#id <- method param “id”

```
@PreAuthorize("hasRole('ADMIN') or #username == authentication.name")
public void updateUser(String username) { ... }
```

© Pedro Alves 2026

Spring

Model-based access control

Capabilities encoded in annotations

Statically identify capabilities and associated rules in annotations

```
@PutMapping("/posts/{id}")
@AllowedForReplaceBook
fun update(@PathVariable id: Long, @RequestBody updated: BlogPost):
      ResponseEntity<BlogPost> {
    ...
}
```

© Pedro Alves 2026

Spring

Model-based access control

Capabilities encoded in annotations

```
@Target(AnnotationTarget.FUNCTION)
@Retention(AnnotationRetention.RUNTIME)
@PreAuthorize("@postService.isAuthor(#id, authentication.name)")
annotation class AllowedForReplaceBook
```

```
@PutMapping("/posts/{id}")
@AllowedForReplaceBook
fun update(@PathVariable id: Long, @RequestBody updated: BlogPost):
      ResponseEntity<BlogPost> {
    ...
}
```

© Pedro Alves 2026

Spring

Model-based access control

Capabilities encoded in annotations

- ✅ Doesn’t pollute the controller/business logic with authorization checks
- ✅ Declares the rules instead of programming them
- ❌ No compile-time checking (SPEL is evaluated in runtime)
- ✅ Improves reuse (if the same rule is applied in multiple places)

© Pedro Alves 2026

Exercise

	Embed checks in controller/service code	@PreAuthorize in controller	@PreAuthorize in service	Capabilities encoded in annotations
Style (declarative, imperative)				
How are rules defined? (code, SpEL)				
Rules flexibility (high, medium, low)				
Reuse level (high, medium, low)				
Main benefit				

© Pedro Alves 2026

Solution

	Embed checks in controller/service code	@PreAuthorize in controller	@PreAuthorize in service	Capabilities encoded in annotations
Style (declarative, imperative)	imperative	declarative	declarative	declarative
How are rules defined? (code, SpEL)	code	SpEL	SpEL	code or SpEL
Rules flexibility (high, medium, low)	high	medium	medium	high
Reuse level (high, medium, low)	low	medium	high	high
Main benefit	full control	early check	close to business rule	clear separation of concerns

© Pedro Alves 2026